

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ
НАПРАВЛЕНИЕ ПРОГРАММНАЯ ИНЖЕНЕРИЯ
ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА СИСТЕМНОЕ И ПРИКЛАДНОЕ
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

ОТЧЕТ ПО РЕШЕНИЮ ЗАДАЧ БЛОКА №1
курса «Алгоритмы и структуры данных»

Выполнил студент:

Исаева А.А.

группа: Р3209

Преподаватель:

Косяков М. С.,

Смирнов В. И.

Санкт-Петербург, 2026 г.

Содержание

1. Задача 1	3
1. Листинг кода	3
2. Описание	4
3. Обоснование	4
4. Сложность	5
2. Задача 2	5
1. Листинг кода	5
2. Описание	7
3. Обоснование	7
4. Сложность	8
3. Задача 3	8
1. Листинг кода	8
2. Описание	9
3. Обоснование	10
4. Сложность	10
4. Задача 3	10
1. Листинг кода	10
2. Описание	11
3. Обоснование	12
4. Сложность	12
5. Вывод	12

1. Задача 1

1. 1. Листинг кода

```
1 #include <cstring>
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 int main() {
7     int n;
8     int ai, ai1, ai2;
9     int first_index_max = 1, last_index_max = 1;
10    int first_index_current = 1, last_index_current =
        1;
11
12    cin >> n;
13    cin >> ai >> ai1;
14
15    for (int i = 3; i <= n; i++) {
16        cin >> ai2;
17
18        if (ai2 == ai1 && ai2 == ai) {
19            last_index_current = i - 1;
20
21            if (last_index_current - first_index_current >
                last_index_max - first_index_max) {
22                last_index_max = last_index_current;
23                first_index_max = first_index_current;
24            }
25            first_index_current = i - 1;
26        }
27        ai = ai1;
28        ai1 = ai2;
29    }
30
31    if (n - first_index_current > last_index_max -
        first_index_max) {
32        cout << first_index_current << " " << n;
33    } else {
34        cout << first_index_max << " " << last_index_max;
35    }
36
```

```

37 return 0;
38 }

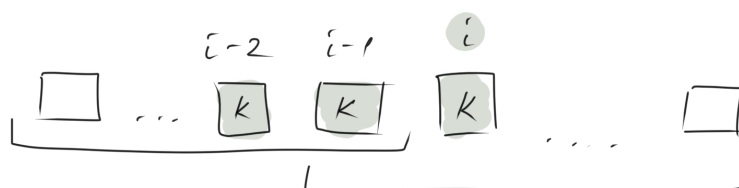
```

Листинг 1: Листинг кода задачи №1

1. 2. Описание

Необходимо найти самый длинный непрерывный подотрезок массива, в котором нет трёх подряд идущих одинаковых элементов. Для этого выполняется проход по массиву, при обнаружении трех одинаковых элементов подряд, текущий отрезок завершается, и начинается новый отрезок со второго элемента этой тройки. В процессе запоминается самый длинный из найденных отрезков.

- Вводятся первые два элемента последовательности, для каждого следующего элемента проверяется, образуют ли текущий, предыдущий и позапрошлый элементы тройку одинаковых.
- Если тройка найдена, текущий допустимый отрезок заканчивается на индексе предыдущего элемента, его длина сравнивается с длиной наибольшего ранее найденного отрезка, если текущий больше, обновляется. Начало нового отрезка устанавливается на индекс предыдущего элемента.
- После завершения цикла проверяется последний незавершённый отрезок (от текущего начала до конца массива) и если длина больше максимального из цикла, он становится лучшим.
- Выводятся границы лучшего отрезка.



1. 3. Обоснование

- Текущий отрезок от `first_index_current` до текущей позиции не содержит трёх подряд одинаковых.
- При обнаружении тройки текущий отрезок не может быть продолжен, поэтому его конец фиксируется. Новый отрезок начинается со второго элемента тройки, что гарантирует отсутствие тройки в его начале.
- Все возможные максимальные отрезки будут рассмотрены, поскольку их границы определяются либо тройками, либо концом массива.

Выбор самого первого при равенстве длин обеспечивается порядком обновления максимума: при равенстве длин новый отрезок не заменяет старый.

1. 4. Сложность

Время: $O(n)$ – один проход по массиву.

Память: $O(1)$ – используются только несколько целочисленных переменных.

2. Задача 2

2. 1. Листинг кода

```
1 #include <cctype>
2 #include <cstring>
3 #include <iostream>
4 #include <string>
5 #include <vector>
6 using namespace std;
7
8 bool comparing_letters(char a, char b) {
9     if ((isupper(a) && islower(b)) || (isupper(b) &&
10         islower(a))) {
11         if (tolower(a) == tolower(b)) {
12             return 1;
13         }
14     }
15     return 0;
16 }
17
18 int main() {
19     string s;
20     cin >> s;
21
22     int n;
23     n = s.length();
24
25     vector<int> orders(n / 2);
26
27     vector<pair<char, int>> traps_and_animals;
28
29     int counts_animals = 1;
```

```

29  int counts_traps = 1;
30
31  for (int i = 0; i < n; i++) {
32      if (!traps_and_animals.empty() &&
33          comparing_letters(s[i], traps_and_animals.back().
34          first)) {
35          if (isupper(s[i])) {
36              orders[counts_traps - 1] = traps_and_animals.
37              back().second;
38              counts_traps++;
39          } else {
40              orders[traps_and_animals.back().second - 1] =
41              counts_animals;
42              counts_animals++;
43          }
44          traps_and_animals.pop_back();
45      } else {
46          if (islower(s[i])) {
47              traps_and_animals.push_back({s[i],
48              counts_animals});
49              counts_animals++;
50          } else {
51              traps_and_animals.push_back({s[i],
52              counts_traps});
53              counts_traps++;
54          }
55      }
56  }
57  if (!traps_and_animals.empty()) {
58      cout << "Impossible";
59  } else {
60      cout << "Possible" << endl;
61      for (int i = 0; i < n / 2; i++) {
62          if (i == n / 2 - 1) {
63              cout << orders[i];
64          } else {
65              cout << orders[i] << " ";
66          }
67      }
68      cout << endl;
69  }
70  return 0;
71 }

```

2. 2. Описание

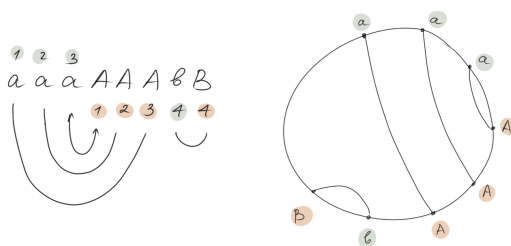
Задача сводится к проверке возможности сопоставить каждое животное с ловушкой того же вида так, чтобы пути не пересекались. Это аналогично проверке правильной скобочной последовательности, где открывающая скобка – животное или ловушка, а закрывающая – соответствующий парный элемент.

Последовательность открывающих и закрывающих скобок называется *правильной скобочной последовательностью*, если она удовлетворяет следующим двум условиям:

- ▶ количества открывающих и закрывающих скобок равны;
- ▶ для любого k среди первых k скобок открывающих не меньше, чем закрывающих.

При обходе строки слева направо для каждого символа проверяется, можно ли его сопоставить с последним элементом массива: это возможно, если они являются парой ловушка-животное. Если да, то последний элемент удаляется из массива, и фиксируется соответствие между номерами, иначе текущий элемент помещается в конец массива.

В процессе ведётся два счётчика: для животных и для ловушек, чтобы присваивать им уникальные номера в порядке появления. При сопоставлении запоминается, какое животное попадает в какую ловушку. В конце, если массив пуст, все пары найдены и пути не пересекаются, иначе решение невозможно.



2. 3. Обоснование

- В массиве хранятся элементы без пары, и их порядок соответствует вложенности: если элемент A находится раньше элемента B , то пара для A должна будет находиться позже, чем для B , что гарантирует непересекаемость путей.
- При обнаружении пары для последнего элемента массива, он корректно сопоставляется, так как текущий элемент является ближайшим.

- Если после прохода всей строки массив не пуст, значит, есть элементы без пары, что делает задачу неразрешимой.
- Порядок вывода требует для каждой ловушки указать номер животного, которое в неё попало. Эти номера сохраняются в массиве `orders`.

2. 4. Сложность

Время: $O(n)$ – один проход по строке, каждая операция с массивом выполняется за $O(1)$.

Память: $O(n)$ – в худшем случае массив может содержать до n элементов, также хранятся номера животных и ловушек.

3. Задача 3

3. 1. Листинг кода

```

1 #include <cctype>
2 #include <iostream>
3 #include <map>
4 #include <string>
5 #include <vector>
6
7 using namespace std;
8
9 bool is_number(string s) {
10     if (s.empty())
11         return false;
12     return isdigit(s[0]) || s[0] == '-';
13 }
14
15 int main() {
16     map<string, vector<int>> parameters;
17     vector<vector<string>> blocks;
18     blocks.push_back(vector<string>());
19
20     string line;
21     while (cin >> line) {
22         if (line == "{") {
23             blocks.push_back(vector<string>());
24         } else if (line == "}") {
25             vector<string> current_block = blocks.back();

```



```

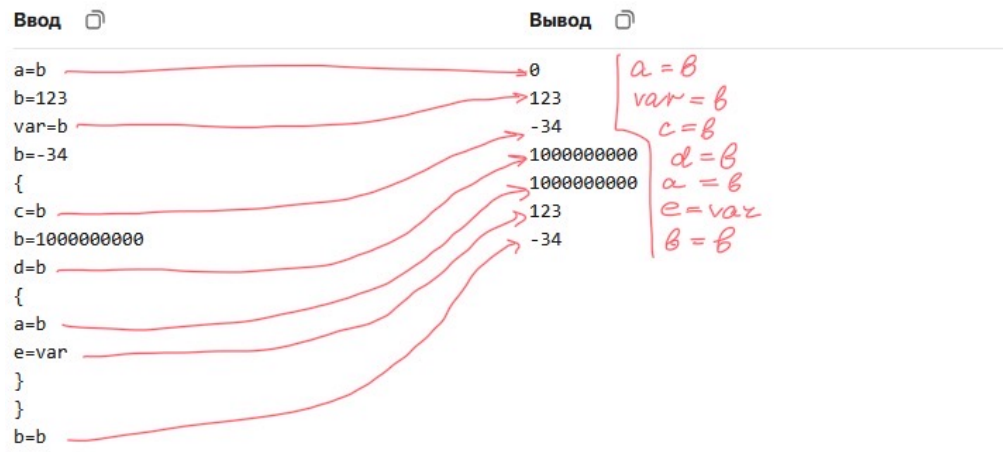
26     for (size_t i = 0; i < current_block.size(); i
    ++ ) {
27         string var_name = current_block[i];
28         parameters[var_name].pop_back();
29     }
30     blocks.pop_back();
31 } else {
32     int eq_pos = line.find('=');
33     string var1 = line.substr(0, eq_pos);
34     string right_part = line.substr(eq_pos + 1);
35     int val = 0;
36     if (is_number(right_part)) {
37         val = stoi(right_part);
38     } else {
39         if (!parameters[right_part].empty()) {
40             val = parameters[right_part].back();
41         }
42         cout << val << "\n";
43     }
44     parameters[var1].push_back(val);
45     blocks.back().push_back(var1);
46 }
47 }
48 return 0;
49 }

```

Листинг 3: Листинг кода задачи №3

3. 2. Описание

Для области видимости переменных используется массив, в котором каждому имени сопоставлен стек значений, где вершина это текущее значение. Для управления блоками применяется стек списков изменённых переменных: при входе в блок добавляется новый список, при выходе для всех переменных из верхнего списка удаляются верхние значения из их стеков.



3. 3. Обоснование

- Изначально все переменные равны 0.
- При присваивании $\langle \text{var1} \rangle = \langle \text{число} \rangle$ число помещается в стек var1 , var1 добавляется в текущий список изменённых.
- При присваивании $\langle \text{var1} \rangle = \langle \text{var2} \rangle$ сначала считывается текущее значение var2 (вершина стека или 0), оно выводится, затем это же значение помещается в стек var1 , и var1 добавляется в текущий список.
- Стек блоков обеспечивает восстановление значений при выходе из блока.

3. 4. Сложность

Время: $O(N \log M)$, где N – число строк, M – число различных переменных (из-за использования map).

Память: $O(N)$ в худшем случае для хранения стеков значений и списков изменённых переменных.

4. Задача 3

4. 1. Листинг кода

```

1 #include <iostream>
2 #include <unordered_map>
3 #include <vector>
4
5 using namespace std;
6
7 int main() {
8     long long a, b, c, d, k;
```

```

9  cin >> a >> b >> c >> d >> k;
10 unordered_map<long long, long long> seen_numbers;
11 vector<long long> values;
12 seen_numbers[a] = 0;
13 values.push_back(a);
14 for (long long day = 1; day <= k; ++day) {
15     long long next_a = a * b - c;
16     if (next_a <= 0) {
17         cout << 0;
18         return 0;
19     }
20     if (next_a > d) {
21         next_a = d;
22     }
23     if (day == k) {
24         cout << next_a;
25         return 0;
26     }
27     if (seen_numbers.count(next_a)) {
28         long long start_day = seen_numbers[next_a];
29         long long cycle_len = day - start_day;
30         long long rem_days = k - day;
31         long long pos = start_day + (rem_days %
cycle_len);
32         cout << values[pos];
33         return 0;
34     }
35     seen_numbers[next_a] = day;
36     values.push_back(next_a);
37     a = next_a;
38 }
39 return 0;
40 }

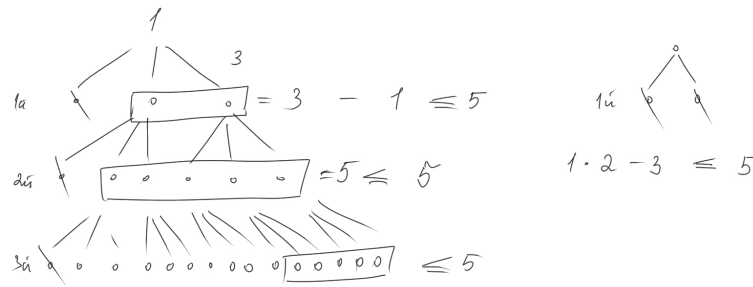
```

Листинг 4: Листинг кода задачи №4

4. 2. Описание

Количество бактерий изменяется по детерминированному правилу и ограничено сверху d (не более 1000). Так как k может быть большим, прямое моделирование невозможно. Последовательность значений обязательно войдёт в цикл не позже чем через $d+1$ шаг. Для обнаружения цикла используется хеш-таблица (`unordered_map`), запоминающая для каждого значения

день его первого появления. При повторении значения находится цикл, и остаток дней вычисляется по модулю его длины.



4. 3. Обоснование

- Конечность состояний гарантирует цикл.
- Хеш-таблица позволяет найти первое повторение за $O(1)$ в среднем.
- При обнаружении цикла оставшиеся дни корректно считаются по модулю.

4. 4. Сложность

Время: $O(d)$ максимальное число шагов до цикла.

Память: $O(d)$ для хранения таблицы и вектора значений.

5. Вывод

В ходе решения задач были применены фундаментальные алгоритмические подходы и структуры данных, рассмотренные в курсе.